ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Е.О. Шумова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

«Наследование классов, базовый класс, производный класс»

по курсу: ОСНОВЫ ПРОГРАММИРОВАНИЯ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2023

1. Условие с вариантом

Вариант №7

Создать класс Board для описания шахматной доски. В нём предусмотреть массив 8x8 элементов и метод для перевода цифр 1-8 в буквы A-H и обратно. На основе класса Board создать класс Composition для составления шахматной композиции. В нём предусмотреть возможность многократного добавления/удаления фигур на доску, распечатку композиции.

2. Листинг программы

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

class Board {
protected:
    char board[8][8];

public:
    Board();
    char numToLetter(int num);
    int letterToNum(char letter);
    void printBoard();
};

class Composition : public Board {
public:
    Composition();
    void addPiece(int row, char col, char piece);
    void removePiece(int row, char col);
    bool isValidPosition(int row, char col);
    void printComposition();
    char numToLetter(int num);
    int letterToNum(char letter);
};

// Определения функций класса Board
Board::Board() {
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            board[i][j] = '-';
        }
    }
}

char Board::numToLetter(int num) {
    if (num >= 1 && num <= 8) {
        return static_cast<char>('A' + num - 1);
    }
    return ' ';
}

int Board::letterToNum(char letter) {
    if (letter >= 'A' && letter <= 'H') {
        return static_cast<int>(letter - 'A' + 1);
    }
    return 0;
}

void Board::printBoard() {
```

```
    cout << "   A B C D E F G H" << endl;
    for (int i = 0; i < 8; i++) {
        cout << i + 1 << "  ";
        for (int j = 0; j < 8; j++) {
            cout << board[i][j] << " ";
        }
        cout << endl;
    }
}

// Определения функций класса Composition
Composition::Composition() : Board() {}

char Composition::numToLetter(int num) {
    if (num >= 1 && num <= 8) {
        return static_cast<char>('A' + num - 1);
    }
    return ' ';
}

int Composition::letterToNum(char letter) {
    if (letter >= 'A' && letter <= 'H') {
        return static_cast<int>(letter - 'A' + 1);
    }
    return 0;
}

void Composition::addPiece(int row, char col, char piece) {
    int colIndex = letterToNum(col) - 1;
    int rowIndex = row - 1;
    if (isValidPosition(row, col) && rowIndex >= 0 && rowIndex < 8 && colIndex >= 0 &&
colIndex < 8) {
        board[rowIndex][colIndex] = piece;
    }
    else {
        cout << "Неверные координаты. Введите корректные координаты шахматной доски." <<
endl;
    }
}

void Composition::removePiece(int row, char col) {
    int colIndex = letterToNum(col) - 1;
    int rowIndex = row - 1;
    if (isValidPosition(row, col) && rowIndex >= 0 && rowIndex < 8 && colIndex >= 0 &&
colIndex < 8) {
        board[rowIndex][colIndex] = '-';
    }
    else {
        cout << "Неверные координаты. Введите корректные координаты шахматной доски." <<
endl;
    }
}

bool Composition::isValidPosition(int row, char col) {
    int colIndex = letterToNum(col) - 1;
    int rowIndex = row - 1;
    return rowIndex >= 0 && rowIndex < 8 && colIndex >= 0 && colIndex < 8;
}

void Composition::printComposition() {
    cout << "Chess Composition:" << endl;
    printBoard();
}

void interpretInput(string input, int& row, char& col, char& piece) {
```

```
    if (input.size() >= 5) {
        if (isdigit(input[0])) {
            row = input[0] - '0';
            col = input[2];
            piece = input[4];
        }
        else {
            col = input[0];
            row = input[2] - '0';
            piece = input[4];
        }
    }
    else if (input.size() >= 3) {
        if (isdigit(input[0])) {
            row = input[0] - '0';
            col = input[2];
            piece = '-';
        }
        else {
            col = input[0];
            row = input[2] - '0';
            piece = '-';
        }
    }
}

int main() {
    system("chcp 1251 > nul");
    system("color 71");
    Composition composition;

    char option;
    do {
        cout << "Выберите действие:" << endl;
        cout << "1. Добавить фигуру" << endl;
        cout << "2. Удалить фигуру" << endl;
        cout << "3. Вывести композицию" << endl;
        cout << "0. Выход" << endl;

        cin >> option;

        switch (option) {
        case '1': {
            int row;
            char col, piece;
            cout << "Введите координаты и тип фигуры (строка, столбец, фигура): ";

            // Считываем координаты как строку
            string input;
            cin.ignore();  // Очищаем буфер ввода
            getline(cin, input);

            // Интерпретируем ввод
            interpretInput(input, row, col, piece);

            composition.addPiece(row, col, piece);
            break;
        }
        case '2': {
            int row;
            char col, piece;
            cout << "Введите координаты и тип фигуры для удаления (строка, столбец): ";

            // Считываем координаты как строку
            string input;
```

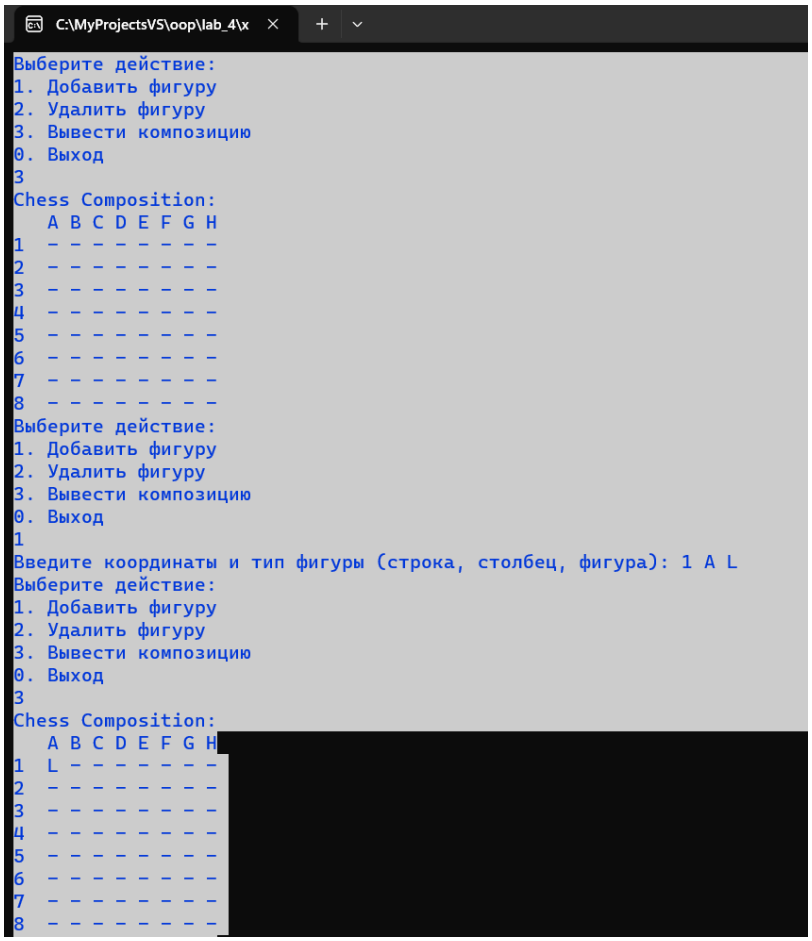
```
            cin.ignore();  // Очищаем буфер ввода
            getline(cin, input);

            // Интерпретируем ввод
            interpretInput(input, row, col, piece);

            composition.removePiece(row, col);
            break;
        }
        case '3':
            composition.printComposition();
            break;
        case '0':
            break;
        default:
            cout << "Неверный ввод. Попробуйте снова." << endl;
            break;
        }
    } while (option != '0');

    return 0;
}
```

3. Скриншоты с исходными данными и выполнением программы



```
Введите координаты и тип фигуры (строка, столбец, фигура): H 8 K
Выберите действие:
1. Добавить фигуру
2. Удалить фигуру
3. Вывести композицию
0. Выход
3
Chess Composition:
   A B C D E F G H
1  L - - - - - - -
2  - - - - - - - -
3  - - - - - - - -
4  - - - - - - - -
5  - - - - - - - -
6  - - - - - - - -
7  - - - - - - - -
8  - - - - - - - K
Выберите действие:
1. Добавить фигуру
2. Удалить фигуру
3. Вывести композицию
0. Выход
```

```
Выберите действие:
1. Добавить фигуру
2. Удалить фигуру
3. Вывести композицию
0. Выход
2
Введите координаты и тип фигуры для удаления (строка, столбец): 1 A
Выберите действие:
1. Добавить фигуру
2. Удалить фигуру
3. Вывести композицию
0. Выход
3
Chess Composition:
   A B C D E F G H
1  - - - - - - - -
2  - - - - - - - -
3  - - - - - - - -
4  - - - - - - - -
5  - - - - - - - -
6  - - - - - - - -
7  - - - - - - - -
8  - - - - - - - K
Выберите действие:
1. Добавить фигуру
2. Удалить фигуру
3. Вывести композицию
0. Выход
```

```
Выберите действие:
1. Добавить фигуру
2. Удалить фигуру
3. Вывести композицию
0. Выход
2
Введите координаты и тип фигуры для удаления (строка, столбец): H 8
Выберите действие:
1. Добавить фигуру
2. Удалить фигуру
3. Вывести композицию
0. Выход
3
Chess Composition:
   A B C D E F G H
1  - - - - - - - -
2  - - - - - - - -
3  - - - - - - - -
4  - - - - - - - -
5  - - - - - - - -
6  - - - - - - - -
7  - - - - - - - -
8  - - - - - - - -
Выберите действие:
1. Добавить фигуру
2. Удалить фигуру
3. Вывести композицию
0. Выход
```

4. Вывод

Я разработал программу для управления шахматной доской и композицией фигур на ней. Программа позволяет пользователю добавлять и удалять фигуры на доске, а также выводить текущую композицию. Так же обеспечил удобный ввод координат фигур и их типов, а также предоставил пользователю простой текстовый интерфейс для взаимодействия с программой. В ней я создал базовый класс Board, представляющий шахматную доску, и определил в нем методы для преобразования координат и отображения доски. Затем я создал производный класс Composition, который наследует функциональность доски и добавил в него методы для управления фигурами на доске, включая добавление и удаление фигур.